

ZeroJ Use Case Demos

ZeroJ Use Case Demos — Presentation

Zero-Knowledge Proofs on Cardano: From Circuit to On-Chain Verification

5 end-to-end demo applications demonstrating real-world ZK use cases

What is ZeroJ?

ZeroJ is a **pure Java ZK toolkit** for Cardano that enables:

- **Circuit definition** in Java (CircuitSpec DSL)
- **Proof generation** using Groth16 on BLS12-381 (pure Java, zero native dependencies)
- **On-chain verification** via Cardano Plutus V3 BLS12-381 builtins
- **Smart contracts** written in Java using Julc (Java → Plutus V3 compiler)

Java Circuit DSL → R1CS → Groth16 Prove (pure Java) → 192-byte proof → Plutus V3 Verify (on-chain)

Technology Stack (All Demos)

Layer	Technology
Backend	Spring Boot 3.5.0-M3, Java 25 (GraalVM)
ZK Proofs	ZeroJ — Groth16, BLS12-381, pure Java prover
On-chain	Julc → Plutus V3 validators
Cardano Client	cardano-client-lib 0.8.0
Frontend	Svelte 5 + Vite
Persistence	H2 (metadata), RocksDB (Merkle trees)
Local Devnet	Yaci DevKit
Proof Caching	SetupCache (binary SRS + setup serialization)

Demo Portfolio Overview

#	Demo	Port	What it Proves	On-Chain Pattern
1	Private NFT Ownership	8085	"I own an NFT from this collection"	Sorted linked list + nullifiers
2	Private Voting	8086	"I'm eligible AND my vote is valid"	Sorted linked list + vote tally
3	Identity/KYC	8087	"I meet KYC requirements (age, country)"	Stateless spending validator
4	Digital Product Passport	8088	"Product meets EU compliance thresholds"	Minting policy (mint-if-compliant)
5	Proof of Reserves	8089	"Reserves >= liabilities, all balances >= 0"	Spending validator (attestation)

Code Metrics

Metric	Value
Total Java source files	70 files
Total Java lines of code	~8,200 lines
Total frontend code (Svelte + TypeScript)	~1,500 lines
On-chain Plutus validators	9 validators across 5 demos
ZK circuits	8 circuits (threshold, membership, inspection, solvency, vote, credential, NFT ownership)
Proof size (all circuits)	192 bytes (constant)

Demo 1: Private NFT Ownership

Port: 8085 | Problem: Proving NFT ownership reveals your wallet address

What it Proves

"I own an NFT from this collection — without revealing which wallet holds it"

Architecture

```
Holder → Generate ZK proof (secretKey, tokenName, Merkle path)
        → Submit to sorted linked list on-chain
        → Nullifier prevents reuse (one-time access)
        → Verifier sees: "someone owns an NFT" – nothing else
```

Key Components

Component	Details
Circuit	<code>NFTOwnershipCircuit</code> — ~10,800 constraints, Poseidon Merkle proof (depth 10)
On-chain	<code>NullifierListValidator</code> (sorted linked list) + <code>ZkProofMintingPolicy</code> (Groth16 BLS12-381)
Pattern	Two minting policies coupled — list validates insertion, ZK policy validates proof
Nullifier	<code>Poseidon(tokenName, contextId)</code> — same NFT + same event = same nullifier

Demo Flow

1. Mint NFTs on Yaci DevKit
2. Register holders → build Merkle snapshot
3. Generate ZK proof (~7-30s)
4. Submit on-chain → nullifier inserted in sorted linked list
5. Double access → rejected (nullifier exists)

What’s Hidden vs What’s Revealed

Hidden	Revealed
Wallet address	“Someone owns an NFT from this collection”
Which specific NFT	Nullifier (one-time token)
Other token holdings	192-byte proof

Demo 2: Private Voting

Port: 8086 | Problem: DAO votes are public (enables vote buying, social pressure)

What it Proves

“I’m an eligible voter AND my vote is valid (0 or 1) — without revealing who I am or how I voted”

Architecture

```
Voter → Prove: "I'm in voter Merkle tree + vote ∈ {0,1}"
  → Nullifier = Poseidon(secretKey, electionId) — one vote per voter
  → Commitment = Poseidon(vote, nullifier) — hides vote
  → Insert on-chain via sorted linked list
  → Tally: decode commitments after election
```

Key Components

Component	Details
Circuit	<code>PrivateVoteCircuit</code> — ~10,800 constraints, voter Merkle proof + vote boolean

Component	Details
On-chain	<code>VoteListValidator</code> (linked list) + <code>VoteZkMintingPolicy</code> (Groth16)
Tally	Off-chain: for each nullifier, compute <code>Poseidon(0, null)</code> and <code>Poseidon(1, null)</code> , match against stored commitment
Test accounts	5 voter accounts auto-created at startup with Yaci DevKit funding

Demo Flow

- 1. Election auto-created with 5 voters
- 2. voter1 votes YES → on-chain tx
- 3. voter2 votes NO → on-chain tx
- 4. voter1 tries again → **double vote rejected**
- 5. Tally: YES: 1, NO: 1

Demo 3: Identity / KYC

Port: 8087 | Problem: KYC requires revealing sensitive personal data

What it Proves

"I have a valid credential AND age >= 18 AND my country is in the approved list — without revealing my age, country, or identity"

Architecture

```
Issuer → Issue Poseidon-signed credential: hash(secret, hash(age, country))
User   → Generate ZK proof: credential valid + age check + country Merkle
        → On-chain: spending validator verifies Groth16 proof
        → Verifier sees: "eligible: YES" — nothing else
```

Key Components

Component	Details
Circuit	<code>CredentialCircuit</code> — ~5,000 constraints, credential hash + 8-bit age comparison + country Merkle
On-chain	<code>CredentialGatedValidator</code> (spending validator, datum-based Groth16)
Pattern	Stateless — no nullifiers, proofs can be reused (ongoing DeFi access)
Credential	Poseidon-signed (shared secret). ADR-0014 documents upgrade path to EdDSA/BBS+ for W3C VC compatibility

Test Users

User	Age	Country	Expected
Alice	25	USA	ELIGIBLE
Bob	30	GBR	ELIGIBLE
Charlie	16	USA	NOT ELIGIBLE (underage)
Diana	22	BRA	NOT ELIGIBLE (country)
Eve	45	JPN	ELIGIBLE

Key Difference

Unlike NFT/voting demos, this is **stateless** — no linked list, no nullifiers. The same proof can be used repeatedly for ongoing DeFi access.

Demo 4: Digital Product Passport (DPP)

Port: 8088 | Problem: EU ESPR mandates product compliance data, but supply chain details are trade secrets

What it Proves

“Product meets carbon/recycled/origin/inspection thresholds — without revealing exact values or supply chain details”

Architecture

```
Manufacturer → Register products in Poseidon MPF (RocksDB, persistent)
  → Generate ZK proofs per claim (carbon, recycled, EU origin, inspections)
  → Mint DPP NFT on Cardano (minting policy verifies Groth16 proof)
  → Duplicate prevention via minted-registry MPF
```

Key Innovations

- 1. **Poseidon MPF (Merkle Patricia Forestry)** - Custom PoseidonCommitmentScheme for CCL's MPF — replaces Blake2b with Poseidon - RocksDB persistence — survives restarts, scales to millions of products - ZK-circuit-verifiable Merkle root
- 2. **Multiple ZK Circuits** | Circuit | Claims | Constraints | |——|——|———| | ComplianceThreshold (GTE) | recycled >= 30% | ~1,700 | | ComplianceThreshold (LTE) | carbon <= 50kg | ~1,700 | | InspectionChain | 3 inspections in order | ~10,100 | | CountryMembership | made in EU | ~3,300 |
- 3. **Mint-if-Compliant Pattern** - On-chain minting policy verifies: manufacturer sig + Groth16 proof + isCompliant=1 + qty=1 - Public inputs in output datum, proof in redeemer (proven pattern) - “Skip off-chain check” checkbox demonstrates on-chain rejection for non-compliant products
- 4. **Full Persistence** - H2 database for product metadata (survives restarts) - SetupCache for SRS + setup (startup: 60s → 14s) - Server-side pagination for large product lists

Demo Scenarios

Product	Carbon	Recycled	EU	Inspections	Result
BAT-SN001 (battery)	7kg ✓	45% ✓	DEU ✓	3/3 ✓	ALL COMPLIANT → NFT minted

Product	Carbon	Recycled	EU	Inspections	Result
BAT-SN003 (battery)	65kg ×	20% ×	DEU ✓	3/3 ✓	NOT COMPLIANT → mint rejected
TEX-B2024-001 (100 t-shirts)	3kg ✓	42% ✓	FRA ✓	—	ALL COMPLIANT
TEX-B2024-003 (500 units)	15kg ✓	5% ×	CHN ×	—	NOT COMPLIANT

Demo 5: Proof of Reserves

Port: 8089 | Problem: Exchanges collapse (FTX) — users can't verify solvency without seeing all balances

What it Proves

“Total reserves >= total liabilities AND all account balances >= 0 — without revealing any individual balance”

Architecture

Exchange → Register accounts with balances (H2)

→ Build Merkle Sum Tree (Poseidon, in-memory)

→ Generate solvency proof (Groth16)

→ Publish attestation on-chain (Plutus V3 validates proof)

User → Verify inclusion: "Is my 500 ADA balance in the tree?"

→ Merkle path verification (off-chain)

Key Innovation: Merkle Sum Tree

Unlike a regular Merkle tree, each node carries a **cumulative sum**:

Standard Merkle: Node = Poseidon(left_hash, right_hash)

[proves inclusion only]

Sum Tree: Node = Poseidon(left_hash, right_hash)

[proves inclusion]

+ sum = left_sum + right_sum

[proves total]

Circuit

Component	Details
<code>SolvencyCircuit</code>	Merkle Sum Tree in-circuit + 64-bit range checks + solvency comparison
Range checks	All balances <code>assertInRange(64)</code> — prevents negative balance cheating
Sum verification	Computed sum must match published total liabilities
Solvency	<code>reserves >= totalLiabilities</code> (64-bit comparison)

Demo Scenarios

Scenario	Reserves	Liabilities	Result
Solvent	10,000 ADA	8,500 ADA	✅ SOLVENT — Groth16 verified on-chain
Insolvent	5,000 ADA	8,500 ADA	❌ INSOLVENT — Plutus validator rejects
User verify	—	—	✅ Alice's 500 ADA correctly included

On-Chain Patterns Summary

Each demo showcases a different Cardano UTXO pattern:

Pattern	Used By	How it Works
Sorted Linked List	NFT, Voting	Each entry is a UTXO. Sorted insertion proves non-existence. Natural concurrency.
Stateless Spending	Identity/KYC	Lock ADA → unlock with ZK proof. No state growth. Proof reusable.
Mint-if-Compliant	DPP	Minting policy gates NFT creation. Groth16 verified on-chain.
Attestation UTXO	Proof of Reserves	Periodic UTXO update with solvency proof. Anyone can read.

Performance

Metric	Value
Proof size	192 bytes (constant for all circuits)
Proof generation (simple circuit, ~1,700 constraints)	3-8 seconds
Proof generation (complex circuit, ~10,800 constraints)	15-30 seconds
On-chain verification cost	~0.3-0.5 ADA per Groth16 pairing check
Startup with SetupCache	~14 seconds (vs ~60s cold start)
Circuit compilation	< 1 second

Security Properties (Across All Demos)

Property	How it's Achieved
Zero knowledge	Groth16 proof reveals nothing about secret inputs

Property	How it's Achieved
Soundness	BLS12-381 pairing check — computationally infeasible to forge
On-chain enforcement	Plutus V3 validators verify proofs — can't bypass by using a different UI
Anti-replay	Nullifiers (NFT, Voting), stateless reuse (KYC), per-product minting (DPP)
Persistence	H2 + RocksDB + SetupCache — survives restarts
Deterministic setup	Same tau → same SRS → same policy IDs across restarts

Running the Demos

Prerequisites

```
sdk install java 25.0.2-graal # Java 25 (GraalVM)
yaci-cli devkit start         # Local Cardano devnet
```

Start Any Demo

```
sdk use java 25.0.2-graal
cd zeroj-usecases/<demo-name>

# Top up admin wallet
curl -X POST http://localhost:10000/local-cluster/api/addresses/topup \
  -H "Content-Type: application/json" \
  -d '{"address": "addr_test1qryvgass5dsrf2kx13vgfz76uhp83kv5lagzcp29tcana68ca5aqa6swlq6llfamln09tal7n5kvt4275ckwedpt4v7q48uhex", "adaAmount": 10000}'

# Build and run
./gradlew clean build -x test
java --enable-native-access=ALL-UNNAMED -jar build/libs/<demo-name>-0.1.0-SNAPSHOT.jar
```

Demo URLs

Demo	URL
NFT Ownership	http://localhost:8085
Private Voting	http://localhost:8086
Identity/KYC	http://localhost:8087
Digital Product Passport	http://localhost:8088
Proof of Reserves	http://localhost:8089

Repository Structure

```
zeroj-usecases/
├── CLAUDE.md           # Shared tech stack NFRs
├── PRESENTATION.md     # This document
└── nft-ownership/      # Demo 1: Private NFT Ownership
```

```

├── README.md
├── DESIGN.md           # Detailed technical design
├── src/main/java/.../nft/
│   ├── circuit/       # ZK circuit definition
│   ├── onchain/       # Julc → Plutus V3 validators
│   ├── service/       # Business logic + Cardano integration
│   └── controller/    # REST API
├── frontend/         # Svelte 5 + Vite
├── private-voting/   # Demo 2: Private Voting
├── identity-kyc/     # Demo 3: Identity/KYC
├── digital-product-passport/ # Demo 4: DPP
├── docs/mpf-architecture.md # Poseidon MPF design
├── ...
└── proof-of-reserves/ # Demo 5: Proof of Reserves

```

Key Takeaways

1. **ZK proofs are practical on Cardano today** — pure Java prover, Plutus V3 BLS12-381 builtins, ~192 byte proofs
2. **Multiple UTXO patterns** demonstrated — sorted linked lists, stateless validators, minting policies, attestation UTXOs
3. **Real-world use cases** — NFT privacy, DAO governance, KYC compliance, EU DPP regulation, exchange solvency
4. **Production path clear** — MPC ceremonies replace dev setup, circuits are the same, on-chain verifiers are identical
5. **Full persistence** — H2 + RocksDB + SetupCache means demos survive restarts and scale realistically

Future Work

- **ADR-0014:** Add BabyJubJub + EdDSA for W3C Verifiable Credential compatible identity proofs
- **Private Token Transfer:** Privacy pool (deposit/withdraw) — Cardano's first mixer
- **Recursive proofs:** Aggregate multiple proofs into one (for DPP supply chains, large reserve proofs)
- **MPF in-circuit verification:** `SignalMpf` circuit gadget for full Poseidon MPF proof verification inside ZK
- **Production MPC:** Integrate Hermez/Perpetual Powers of Tau ceremony outputs

Built with [ZeroJ](#) — Pure Java ZK for Cardano